TCS Agentic AI - March 2026-  Notes

Attendance Sheet
Agentic AI - TCS Training March 2026 - Attendance

Recommended Casing:
PascalCase
  * recommended for classes

camelCase
  * non recommended

kebab-case
  * for folders

snake_case
  * for variables, functions, methods

==========

**Project Repositories**

| | **Repository URL** |
|---|---|
| 1. REST API Project | https://github.com/arunprabu/um-rest-api-mar2026 |
| 2. Langchain Examples | https://github.com/arunprabu/lc-tcs-examples-mar2026 |
| 3. Hybrid Naive Rag without Agent | https://github.com/arunprabu/naive-rag-system-hybrid |
| | |

Weekend Todos

1. Create rest for products-management
2. add product
3. list products
4. get product by id
5. update product by id
6. delete product by id (soft delete)
7. Validate the request.body using Pydantic BaseModel

TechStack: Postgres, SQLAlchemy

uv add sqlalchemy psycopg2-binary asyncpg

======

Notes to Arun: evaluation to trainees will happen on python

```
# Different Gemini models to switch to...

google_genai:gemini-3-pro-preview
google_genai:gemini-2.5-flash
google_genai:gemini-3.1-pro-preview
google_genai:gemini-3.1-flash-lite-preview
```

```
OPENWEATHER_API_KEY=353cd58982938890b72f3712adec5923
```

API URL to hit to get weather:
https://api.openweathermap.org/data/2.5/weather?q={city_name}&appid={API_KEY}&units=metric

```
# Different Gemini models to switch to...
google_genai:gemini-3.1-pro-preview
google_genai:gemini-3.1-flash-lite-preview
google_genai:gemini-3-flash-preview
google_genai:gemini-2.5-flash
google_genai:gemini-2.5-flash-lite
google_genai:gemini-2.5-flash-preview-09-2025
```

https://notepad.pw/AgenticAITCS

https://github.com/arunprabu/langchain-rag-demo-dec2025/blob/main/data/HR_Support_Desk_KnowledgeBase.pdf

https://chatgpt.com/share/69c605cf-1284-8320-8785-df55a155e97e

| Capstone Project Teams | Member 1 | Member 2 | Member 3 | Member 4 | Assigned Capstone Project (Level 1) |
|---|---|---|---|---|---|
| Team 1 | Kenwinshikaa A P | Raghul S | Mithul M | Parasuram S | Insurance Claims Processing & Intelligence System |
| Team 2 | Lokesh | Sundhar K A | Karthick S | Harini P | Personalized Retail Banking & Wealth Advisory System |
| Team 3 | Poovarasi Jemini | Guruprasath D | Arvinth Ram | Sai Prasanth A | Intelligent Credit Risk Assessment & Loan Underwriting System |
| Team 4 | Antony Lidia | Karthiga A V | Sibhiraj Hariharan | Ragavendar K | & Wealth Advisory System |
| Team 5 | Santhosh G | Nandhini T | Venkhat Shridharan | Ravivarma E D | Insurance Claims Processing & Intelligence System |
| Team 6 | Thamizhamudhu | Meher Boddapati | Shweta Darshini P | Sanjana P | Intelligent Credit Risk Assessment & Loan Underwriting System |
| Team 7 | Karthick Ragav R | K Ajay | Ahi M J | | Regulatory Compliance Intelligence System |

| | | | | | |
|---|---|---|---|---|---|
| Team 8 | Jayashree P | Tharankhini K G | Raja Annamalai M | | Regulatory Compliance Intelligence System |

Expectations in Capstone Project Level #1

====

1. api/v1/admin/upload (should support uploading .pdf and .txt files and vectorize)
2. api/v1/query (with input validations)
3. You must have one agent with tool to query through and produce the output
4. Use 1536 dimensions of embeddings
5. UI should be built using streamlit
6. Maintain Conversation History using ConversationBufferMemory of Langchain (for 4 member teams only)
7. The project must be having the recommended structure.
8. Follow all recommended practices
9. For easier debugging, log custom run name, metadata and tags in langsmith [must]
10. Output format in streamlit UI

Q: Tell me about empid 123000?

A: emp id 123000 is John, who is an architect.
[page number: 5 | doc: filename.pdf ]
[confidence score: 0.5]

====

======

src/api/v1/services/query_service.py

—

```
import re
from src.core.db import get_vector_store
import os
from dotenv import load_dotenv
import psycopg
from psycopg.rows import dict_row

load_dotenv()

# PGVector connection string uses SQLAlchemy format: postgresql+psycopg://...
# psycopg.connect needs standard format: postgresql://...
_raw_conn = os.getenv("PG_CONNECTION_STRING", "").replace("postgresql+psycopg",
"postgresql")

# Patterns that signal a precise keyword lookup is needed
_KEYWORD_PATTERNS = [
    r"[A-Z]{2,}-\d{4}-\w+",    # policy/ticket codes: POL-2024-HR-007
    r"\b[A-Z]{2,5}\b",          # short uppercase abbreviations: LTA, CTC, ESI
    r"\d{6,}",                  # long numeric IDs / employee numbers
]
_KEYWORD_RE = re.compile("|".join(_KEYWORD_PATTERNS))


def _detect_mode(query: str) -> str:
    stripped = query.strip()
    # if the keyword patterns match anywhere in the query,
    # we prioritize FTS search to find exact matches
    if _KEYWORD_RE.search(stripped):
        return "keyword"

    # if the query is short (3 words or fewer),
    # we treat it as a hybrid case to balance precision and recall
    if len(stripped.split()) <= 3:
        return "hybrid"
```

```
    # if the query is long and doesn't match keyword patterns,
    # we assume it's a natural language question best served by vector search
    return "vector"


# vector search
def query_documents(query: str, k: int = 5) -> list[dict]:
    mode = _detect_mode(query)

    if mode == "keyword":
        return fts_search(query, k=k)

    if mode == "hybrid":
        return _hybrid_search(query, k=k)

    # vector — long natural-language question
    vector_store = get_vector_store()
    docs = vector_store.similarity_search(query, k=k)
    return [{"content": doc.page_content, "metadata": doc.metadata} for doc in docs]

# fts search
def fts_search(query: str, k: int = 5, collection_name: str = "hr_support_desk") ->
list[dict]:
    """
    Keyword search against stored chunks using PostgreSQL tsvector / tsquery / ts_rank.

    Args:
        query:           User query string (plain text, any format)
        k:               Number of top results to return
        collection_name: PGVector collection to search

    Returns:
        List of dicts with 'content', 'metadata', and 'fts_rank'
    """
    sql = """
        SELECT
            e.document                                                  AS content,
            e.cmetadata                                                 AS metadata,
            ts_rank(
                to_tsvector('english', e.document),
                plainto_tsquery('english', %(query)s)
```

```
            )                                                       AS fts_rank
        FROM  langchain_pg_embedding  e
        JOIN  langchain_pg_collection c ON c.uuid = e.collection_id
        WHERE c.name = %(collection)s
          AND to_tsvector('english', e.document)
              @@ plainto_tsquery('english', %(query)s)
        ORDER BY fts_rank DESC
        LIMIT %(k)s;
    """
    with psycopg.connect(_raw_conn, row_factory=dict_row) as conn:
        with conn.cursor() as cur:
            cur.execute(sql, {"query": query, "collection": collection_name, "k": k})
            rows = cur.fetchall()

    return [
        {
            "content":  row["content"],
            "metadata": row["metadata"],
            "fts_rank": round(float(row["fts_rank"]), 4),
        }
        for row in rows
    ]

# hybrid search
def _hybrid_search(query: str, k: int = 5) -> list[dict]:
    """
    Merge vector and FTS results using Reciprocal Rank Fusion (RRF).

    RRF score for a chunk = sum of 1/(rank + 60) across both result lists.
    Chunks appearing in both lists score higher than those in only one.
    The constant 60 prevents top-ranked outliers from dominating.
    """
    vector_store = get_vector_store()
    vector_docs = vector_store.similarity_search(query, k=k)
    fts_docs    = fts_search(query, k=k)

    rrf_scores: dict[str, float] = {}
    chunk_map:  dict[str, dict]  = {}

    for rank, doc in enumerate(vector_docs):
        key = doc.page_content[:120]
        rrf_scores[key] = rrf_scores.get(key, 0) + 1 / (60 + rank + 1)
```

```
        chunk_map[key]  = {"content": doc.page_content, "metadata": doc.metadata}

    for rank, item in enumerate(fts_docs):
        key = item["content"][:120]
        rrf_scores[key] = rrf_scores.get(key, 0) + 1 / (60 + rank + 1)
        chunk_map[key]  = {"content": item["content"], "metadata": item["metadata"]}

    ranked = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)
    return [chunk_map[key] for key, _ in ranked[:k]]
```

**========**

Gitrepo url: https://github.com/arunprabu/naive-rag-system-hybrid

```
LLM Observability Tools
===
 1. LangSmith
   1.1 LangSmith Observability
   1.2 LangSmith Studio (Agent Builder -- NOT TO LEARN NOW)
 2. LangFuse
 3. Arise Phoenix
 4. HeliCone

=======
3 important params to evaluate AI agents
---
 1. Accuracy
 2. Cost
 3. Latency


======
```

Capstone Project PDFs (Download the following files as PDF)

1. Regulatory Compliance Intelligence System

   https://docs.google.com/document/d/1UJ5Q3Up1rXNQnDcu_AeEbwvF8fi5BMCpdpunm1Yfpg8/edit?usp=sharing

2. Personalized Retail Banking & Wealth Advisory System

   https://docs.google.com/document/d/1FAXw2k4JPtoBVvpP8ckIjkNxquS-ratsKwppVawyM0Y/edit?usp=sharing

3. Intelligent Credit Risk Assessment & Loan Underwriting System

   https://docs.google.com/document/d/1kAKuNghMlGWPgn5Lm5PZ2F1PTzouY2dDhEiYU6gJswQ/edit?usp=sharing

4. AI-Powered Insurance Claims Processing & Intelligence System

   https://docs.google.com/document/d/1Vvx6Jq5NUIf8yluRMTOQfBnPTATT-NmWU3YIYXxCh4o/edit?usp=sharing

===

## example14.py

—

```python
import os
import uuid
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

session_id = str(uuid.uuid4())

print("🤖 Chatbot started. Type 'exit' to quit.\n")

messages = []

while True:
    user_input = input("You: ")

    if user_input.lower() == "exit":
        print("👋 Goodbye!")
        break

    messages.append(("human", user_input))

    response = llm.invoke(
        messages,
        config={
            "run_name": "terminal_chat",
            "tags": ["chatbot", "terminal", "v1.2"],
            "metadata": {
                "user_id": "user_001",
                "session_id": session_id,
                "interface": "cli"
            }
        }
    )

    messages.append(("ai", response.content))
```

```
print(f"Bot: {response.content}\n")
```